Introducción a GitLab Runner y CI

Estrategias de Automatización y Pipelines de Próxima Generación

EL CICLO DE INTEGRACIÓN CONTINUA

La CI permite que múltiples desarrolladores integren código en un repositorio compartido de forma frecuente, activando procesos automáticos.

- Detección temprana de errores.
- Reducción del tiempo de despliegue.
- Consistencia en los entornos.

¿QUÉ ES GITLAB RUNNER?

100%

Motor de Ejecución

Es un agente ligero escrito en **Go** que se encarga de ejecutar los trabajos (jobs) definidos en el archivo .gitlab-ci.yml.

No procesa código directamente en el servidor de GitLab; lo hace en máquinas separadas para mayor seguridad y escalabilidad.

ARQUITECTURA DEL SISTEMA

GitLab Server

El cerebro. Coordina pipelines, gestiona el repositorio y envía instrucciones a los runners.

API (Polling)

El Runner consulta constantemente al servidor vía HTTPS para saber si hay trabajos pendientes.

Runner Agent

La unidad de ejecución. Descarga el código, ejecuta scripts y devuelve los logs/artefactos.

SECCIÓN 01

Instalación y Configuración

MÉTODOS DE INSTALACIÓN

Paquetes Nativos: Uso de apt o yum para distribuciones Linux.

Binario Directo: Descarga del archivo compilado para cualquier arquitectura.

Contenedores: Imagen oficial en Docker Hub para despliegues rápidos.

Helm Chart: Instalación en clusters de Kubernetes.

REGISTRO: TOKENS DE AUTENTICACIÓN

Paso 1: Obtener

Generar un "Authentication Token" desde la interfaz de GitLab (Admin o Project Settings).

Paso 2: Registrar

Ejecutar gitlab-runner register e introducir la URL y el Token.

Paso 3: Verificar

Confirmar en la UI que el Runner aparece en verde y con el estado "Online".

ELECCIÓN DEL EJECUTOR (EXECUTOR)

Shell

Simple. Ejecuta comandos directamente en el host. Difícil de limpiar.

Docker

Entornos aislados. Limpieza automática tras cada job. El estándar actual.

Kubernetes

Escalado dinámico. Crea pods bajo demanda para ejecutar tareas pesadas.

SECCIÓN 02

Estructura de Pipelines

STAGES VS JOBS

Stages (Etapas)

Definen el **CUÁNDO**. Representan la secuencia lógica (Build -> Test -> Deploy).

Los trabajos en una misma etapa se ejecutan en paralelo.

Jobs (Trabajos)

Definen el **QUÉ**. Son las tareas específicas (compilar código, ejecutar unit tests).

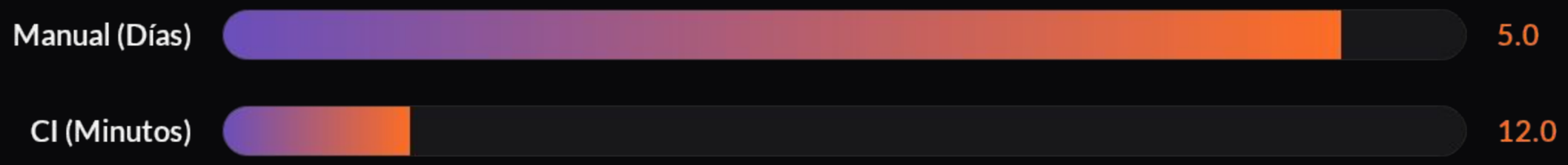
Requieren un ejecutor disponible para procesarse.

ANATOMÍA DE .GITLAB-CI.YML

```yaml
stages:
  - build
  - test

compile_job:
  stage: build
  script:
    - echo "Compilando..."
    - make build
```

IMPACTO DE PRUEBAS AUTOMÁTICAS



"La automatización reduce el tiempo de feedback de días a minutos."

ARTEFACTOS VS CACHÉ

Caché: Optimiza el tiempo descargando dependencias (ej. node_modules) solo cuando cambian.

Artefactos: Archivos generados que deben pasarse entre etapas (ej. un .exe o .jar) o descargarse.

Expiración: Controla cuánto tiempo se guardan para no saturar el almacenamiento.

CALIDAD Y SEGURIDAD

SAST

Análisis estático de seguridad para encontrar vulnerabilidades en el código fuente.

Unit Testing

Verificación de lógica interna. Integración con reportes JUnit.

Code Quality

Detección de "Code Smells" y complejidad ciclomática excesiva.

NOTIFICACIONES EN TIEMPO REAL

Mantén al equipo informado mediante integraciones nativas:

- **Slack/Discord:** Alertas sobre fallos en producción.
- **Email:** Reportes diarios de estado.
- **Webhooks:** Dispara acciones en sistemas externos.

DESPLIEGUE MULTI-CLOUD

GitLab Runner se integra nativamente con:

AWS: Despliegue en EC2, S3 o Lambda.

Google Cloud: Integración con GKE.

Azure: Pipelines de despliegue para Web Apps.